

Fake News Detection Using BERT

Project Description Report

UCS551 Conversational AI

Submitted by:

(102316081) TEJVIR SINGH GREWAL

(102316042) UJJWAL THAPA

(102486013)SHAURYA PUNJ

B.E. Third Year (June-Dec, 2025)

Submitted to:

Dr. Jasmeet Singh & Dr. Simran Setia



Computer Science and Engineering Department

TIET, Patiala

INDEX

S. No.	Title	Page No.
1	Introduction	3
2	Dataset Collection	
3	Data Pre-processing	4
4	Deep learning model description	
5	Evaluation Metrics Used	5
6	Experimental Results	
7	Exploratory Data Analysis	6
8	Sample Predictions	9
9	Working Prototype Demonstration	
10	Code	10
11	Conclusion	33
12	References	33

1. Introduction

With the rapid growth of digital media and online news platforms, the spread of fake news has become a serious global issue. Fake news refers to false or misleading information presented as legitimate news with the intent to deceive readers. Such misinformation can influence public opinion, create social unrest, and affect political and economic decisions. Manual verification of news articles is time-consuming and impractical at large scales, making automated fake news detection an important research problem.

Recent advancements in Natural Language Processing (NLP) and deep learning have enabled machines to understand textual data more effectively. Transformer-based models, especially BERT (Bidirectional Encoder Representations from Transformers), have achieved state-of-the-art performance in many NLP tasks due to their ability to capture contextual relationships between words.

In this project, a fake news detection system is developed using the BERT model to classify news articles as Fake or Real. The model is trained and evaluated using a publicly available dataset, and its performance is analyzed using multiple evaluation metrics and visualizations.

The main objectives of this project are:

1. To understand the problem of fake news detection and its challenges.
2. To build a binary text classification system using deep learning.
3. To fine-tune a pre-trained BERT model for fake news classification.
4. To evaluate the model using standard machine learning metrics.
5. To analyze and visualize the dataset and model predictions.
6. To demonstrate the effectiveness of transformer-based NLP models.

2. Dataset Collection

2.1 Dataset Description

The dataset used in this project consists of two CSV files:

- **Fake.csv** – Contains fake news articles
- **True.csv** – Contains real news articles

Each file includes textual information such as the news title or article content. A binary label is assigned to each article:

- **0** → **Fake News**
- **1** → **Real News**

The two datasets are merged into a single dataset and shuffled before training.

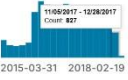
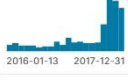
2.2 Dataset Download Link

The dataset was obtained from Kaggle and can be downloaded from the following link:

<https://www.kaggle.com/clmentbisailon/fake-and-real-news-dataset>

2.3 Dataset Statistics

- Total samples: Approximately 44,000 news articles
- Classes: Fake and Real
- Data split:
 - Training set: 64%
 - Validation set: 16%
 - Test set: 20%

Fake.csv (62.79 MB)				True.csv (53.58 MB)			
Detail Compact Column				Detail Compact Column			
About this file				About this file			
fake news				true news			
title	text	subject	date	title	text	subject	date
title of news article	body text of news article	subject of news article	publish date of news article	title of news article	body text of news article	subject of news article	publish date of news article
17903 unique values	[empty] 3% AP News The regu... 0% Other (22851) 97%	News 39% politics 29% Other (7590) 32%		20826 unique values	21192 unique values	politicsNews 53% worldnews 47%	
Donald Trump Sends Out Embarrassing New Year's Eve Message; This is Disturbing	Donald Trump just couldn't wish all Americans a Happy New Year and leave it at that. Instead, he had...	News	December 31, 2017	As U.S. budget fight looms, Republicans flip their fiscal script	WASHINGTON (Reuters) - The head of a conservative Republican faction in the U.S. Congress, who voted...	politicsNews	December 31, 2017
Drunk Bragging Trump Staffer Started Russian Collusion Investigation	House Intelligence Committee Chairman Devin Nunes is going to have a bad day. He's been under the as...	News	December 31, 2017	U.S. military to accept transgender recruits on Monday; Pentagon	WASHINGTON (Reuters) - Transgender people will be allowed for the first time to enlist in the U.S. m...	politicsNews	December 29, 2017

Screenshot of the dataset files (Fake.csv and True.csv) or a snapshot showing dataset loading and shape output.

3. Data Pre-processing

The dataset was first loaded and only the relevant fields, namely the news text and its corresponding label (fake or real), were selected. The categorical labels were then encoded into numerical form to make them compatible with model training. The data was split into training and testing sets to enable unbiased evaluation. A pretrained BERT tokenizer was used to tokenize the news text, during which special tokens such as [CLS] and [SEP] were added, and the text was converted into input IDs and attention masks. To ensure uniform input size, all sequences were padded or truncated to a fixed maximum length. Finally, the processed inputs and labels were converted into PyTorch tensors and organized into DataLoaders for efficient batch-wise training and testing, with minimal manual text cleaning as BERT inherently handles contextual language understanding.

4. Deep Learning Model Description

This project implements the BERT (bert-base-uncased) model, which is a transformer-based pre-trained language model. BERT reads text in both forward and backward directions, allowing it to understand context more effectively than traditional models.

Only BERT was implemented in this project due to its superior performance compared to traditional machine learning and recurrent neural network models for text classification tasks.

4.1 Tokenization

The BERT tokenizer converts text into tokens using WordPiece tokenization. Each input text is:

- Truncated or padded to a fixed maximum length
- Converted into input IDs and attention masks
- Prepared in a format suitable for the BERT model

4.2 Training Process

The model is fine-tuned using:

- **Optimizer:** AdamW
- **Loss Function:** Cross-Entropy Loss
- **Learning Rate Scheduler:** Linear scheduler with warm-up
- **Number of Epochs:** 3

During training, the model parameters are updated using backpropagation. Validation is performed after each epoch to monitor model performance.

```

... Epoch 1: Train loss=0.1277, acc=0.9508
    Val   : loss=0.0496, acc=0.9825
    Epoch 2: Train loss=0.0253, acc=0.9913
    Val   : loss=0.0397, acc=0.9861
    Epoch 3: Train loss=0.0069, acc=0.9978
    Val   : loss=0.0455, acc=0.9887

```

Screenshot showing epoch-wise training and validation loss/accuracy output.

5. Evaluation Metrics Used

To evaluate the performance of the fake news detection model, the following metrics are used:

1. **Accuracy** – Measures the overall correctness of predictions.
2. **Precision** – Measures how many predicted real news articles are actually real.
3. **Recall** – Measures how many real news articles are correctly identified.
4. **F1-Score** – Harmonic mean of precision and recall.
5. **Confusion Matrix** – Displays correct and incorrect classifications.
6. **ROC Curve and AUC Score** – Evaluates performance across different thresholds.
7. **Precision-Recall Curve** – Useful for analyzing class-wise performance.

6. Experimental Results

After training and testing the BERT model on the dataset, strong classification performance was achieved.

6.1 Quantitative Results

- High test accuracy (above 90%)
- High precision, recall, and F1-score for both classes
- Low number of misclassifications

Test accuracy: 0.9907572383073496

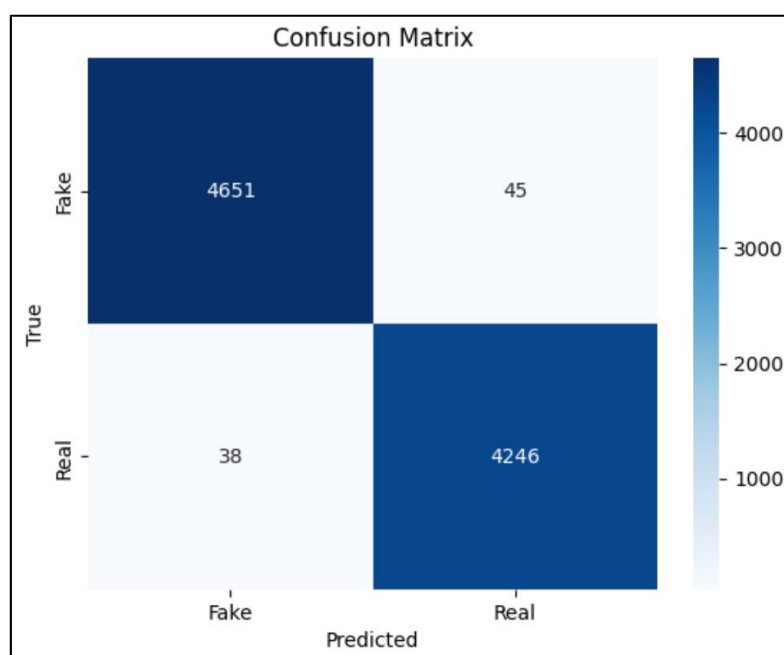
Classification report:

	precision	recall	f1-score	support
0	0.9919	0.9904	0.9912	4696
1	0.9895	0.9911	0.9903	4284
accuracy			0.9908	8980
macro avg	0.9907	0.9908	0.9907	8980
weighted avg	0.9908	0.9908	0.9908	8980

Screenshot of the classification report showing precision, recall, F1-score, and accuracy.

6.2 Confusion Matrix Analysis

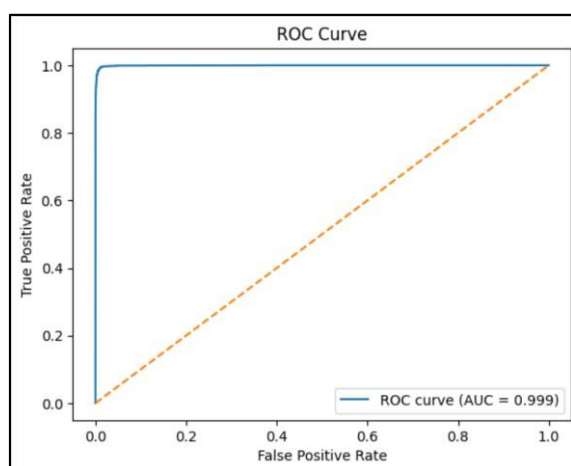
The confusion matrix shows that most fake and real news articles are correctly classified. Very few fake articles are misclassified as real and vice versa, indicating robust model performance.



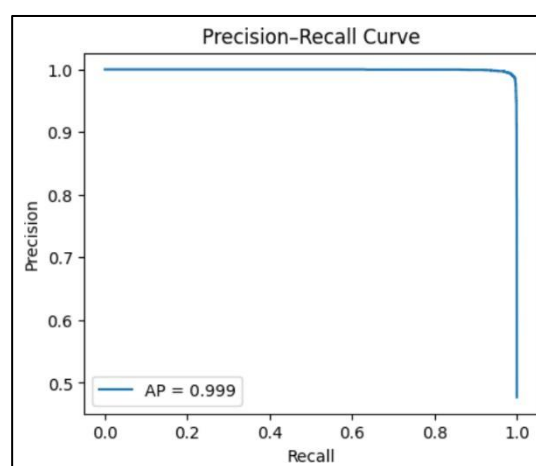
Screenshot of the confusion matrix.

6.3 ROC and Precision–Recall Analysis

The ROC curve shows a high Area Under the Curve (AUC), indicating strong separability between fake and real news. The precision–recall curve further confirms balanced performance across both classes.



Screenshot of the ROC curve



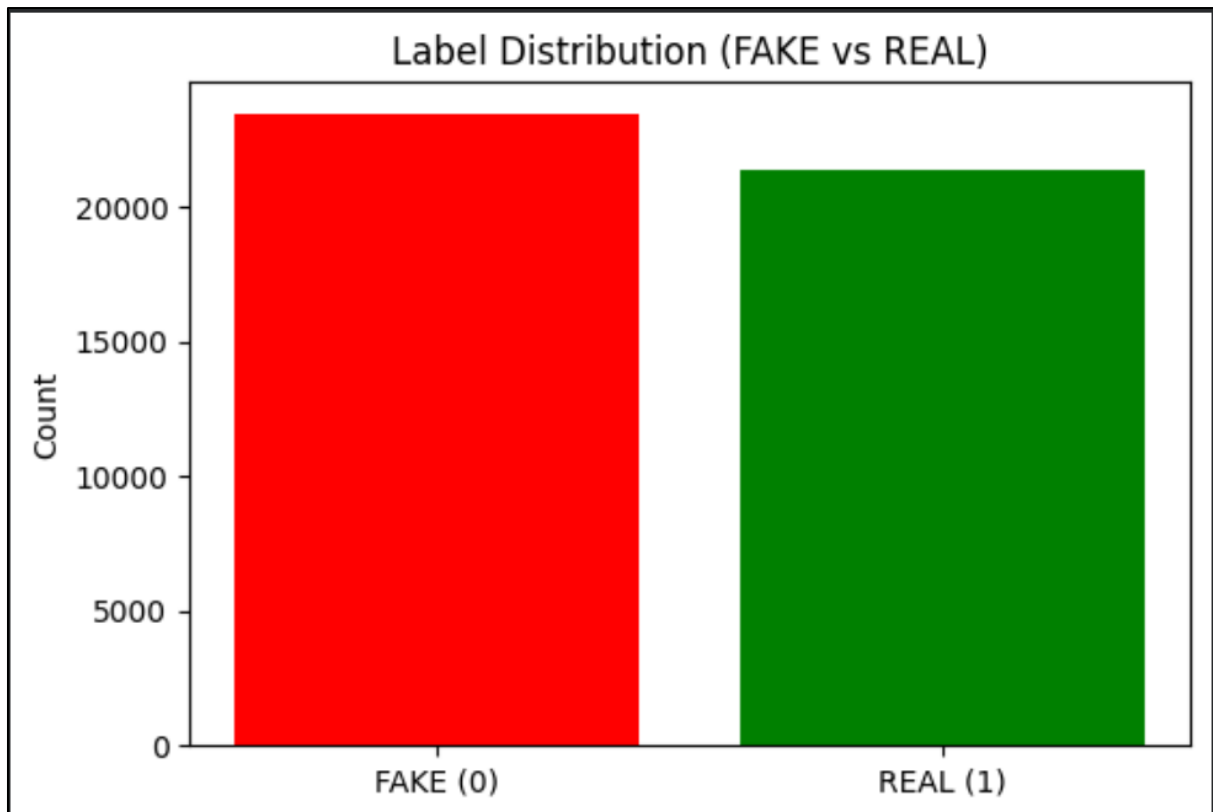
Screenshot of the Precision–Recall curve

7. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to understand the dataset characteristics.

7.1 Label Distribution

The dataset contains a balanced number of fake and real news articles.

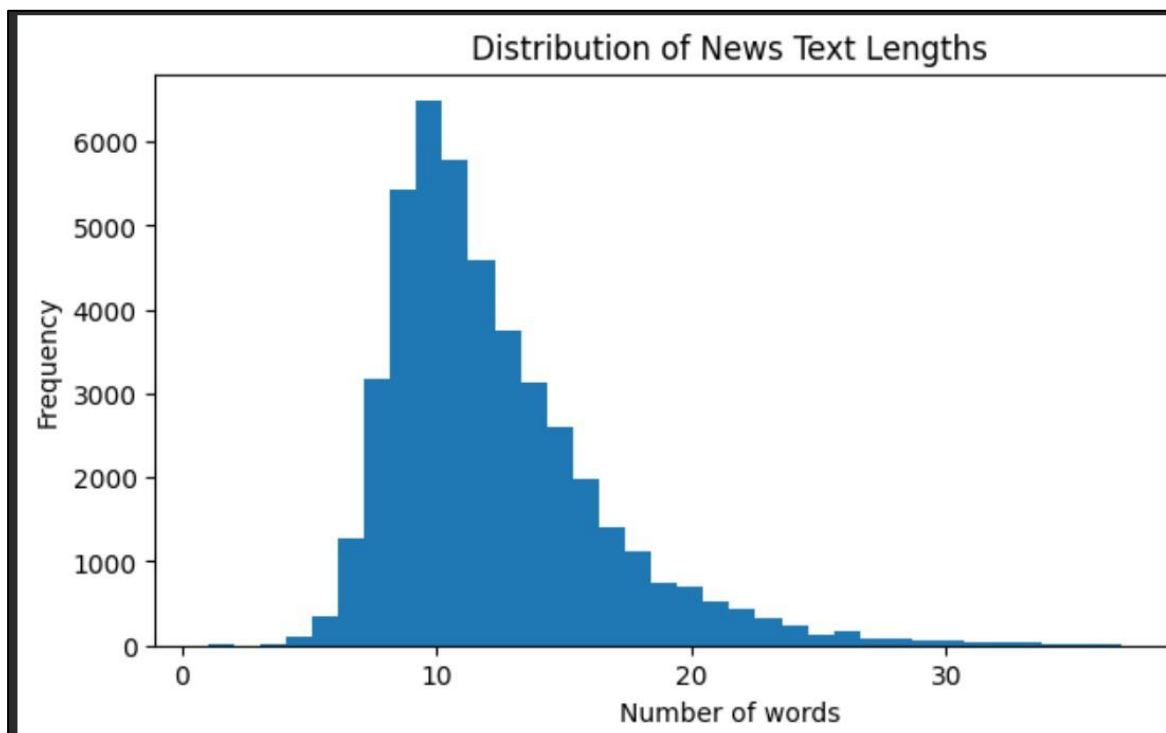


Screenshot of the label distribution bar chart.

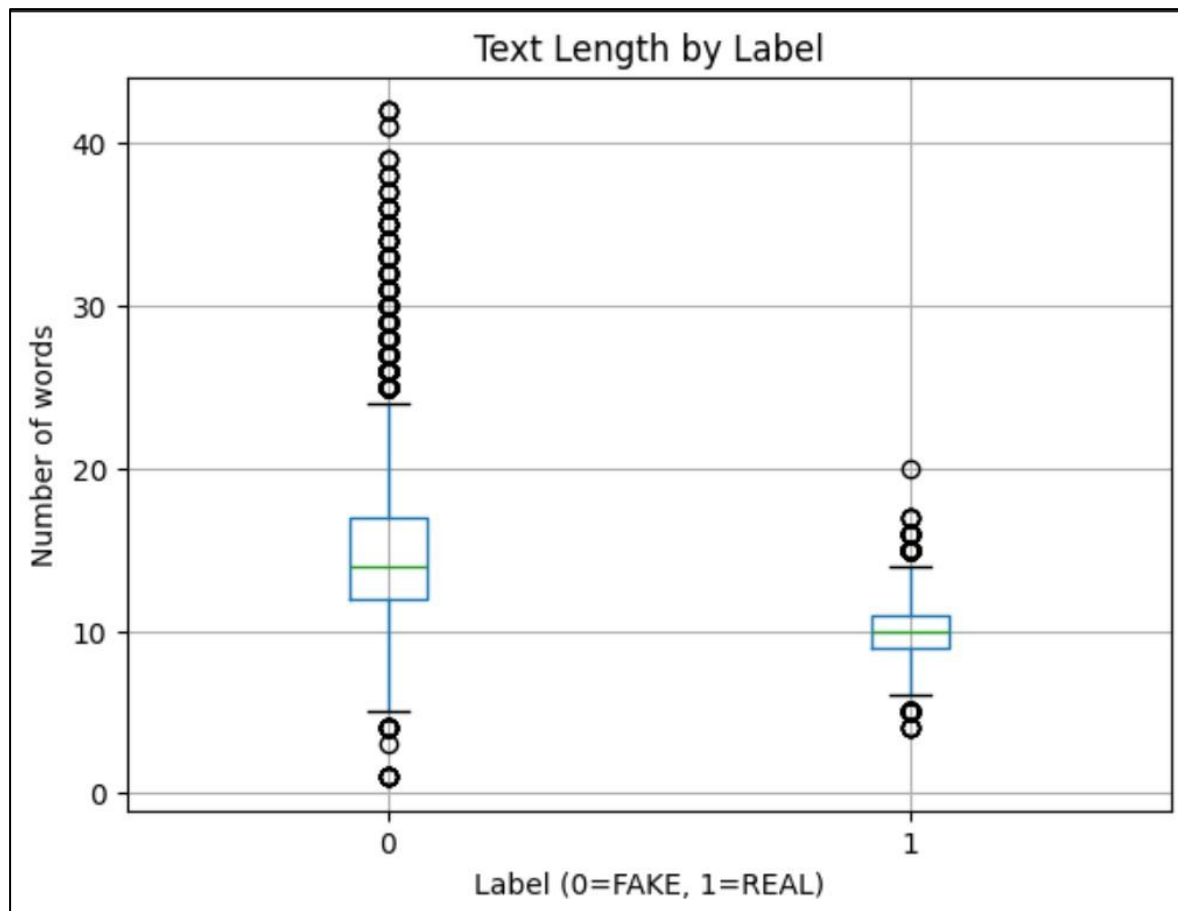
7.2 Text Length Analysis

The distribution of text lengths shows variation between fake and real news articles, which helps the model learn different writing patterns.

- *Histogram of text length distribution*



- *Box plot comparing text length by label*

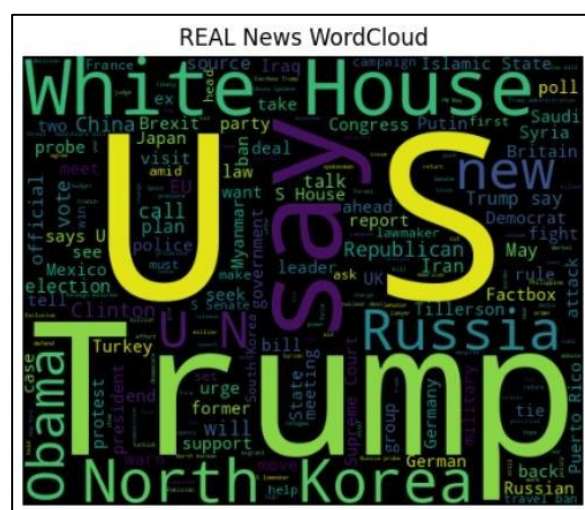


7.3 Word Cloud Analysis

Word clouds reveal commonly used words in fake and real news articles, highlighting stylistic and vocabulary differences.



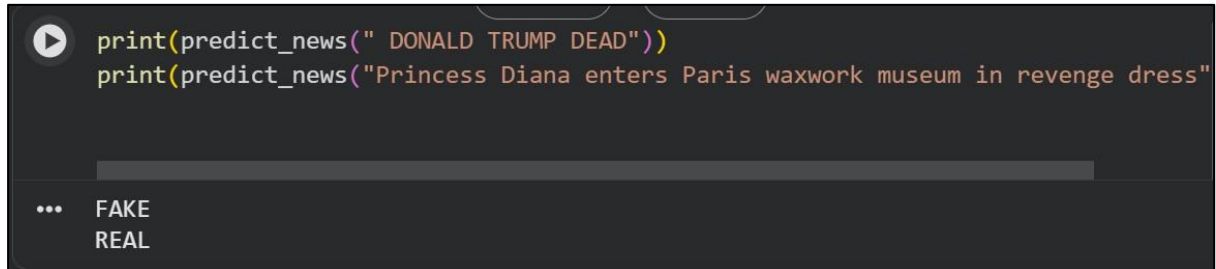
Fake news word cloud



Real news word cloud

8. Sample Predictions

The trained model is tested on custom input sentences to demonstrate real-time prediction capability.



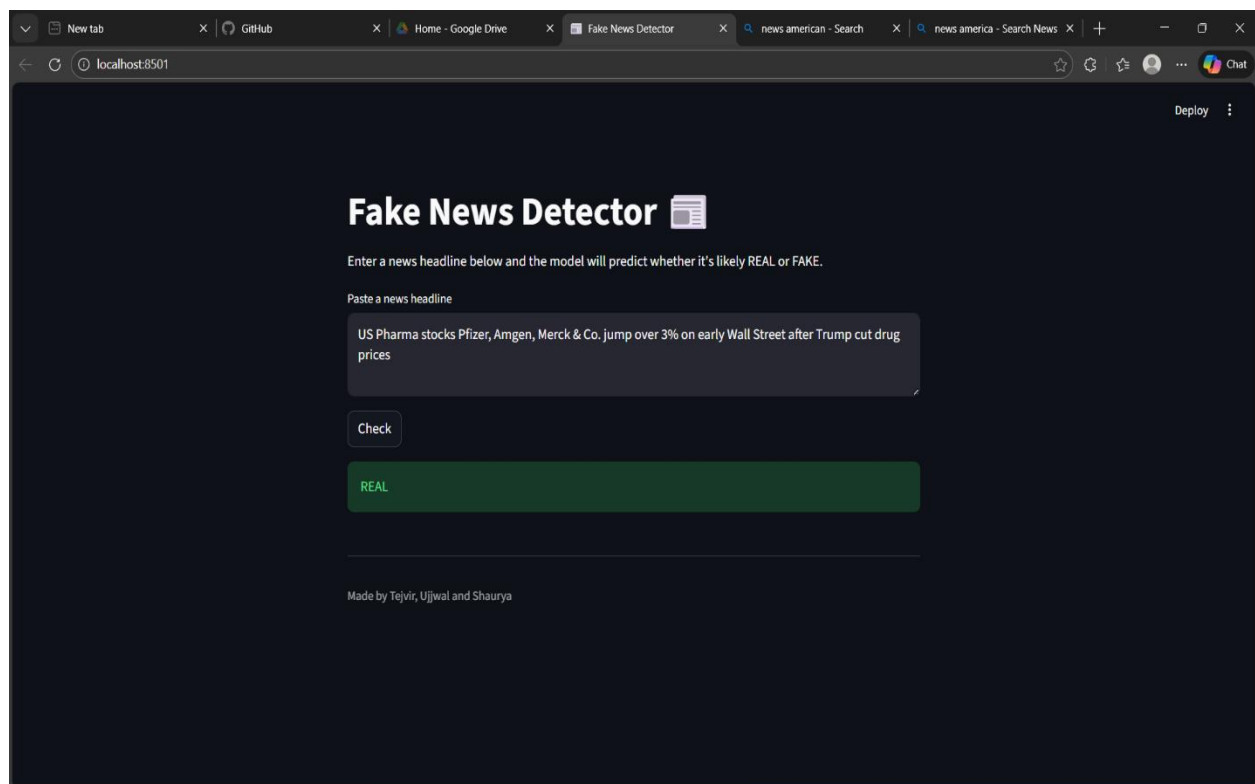
```
print(predict_news(" DONALD TRUMP DEAD"))
print(predict_news("Princess Diana enters Paris waxwork museum in revenge dress"))
```

... FAKE
REAL

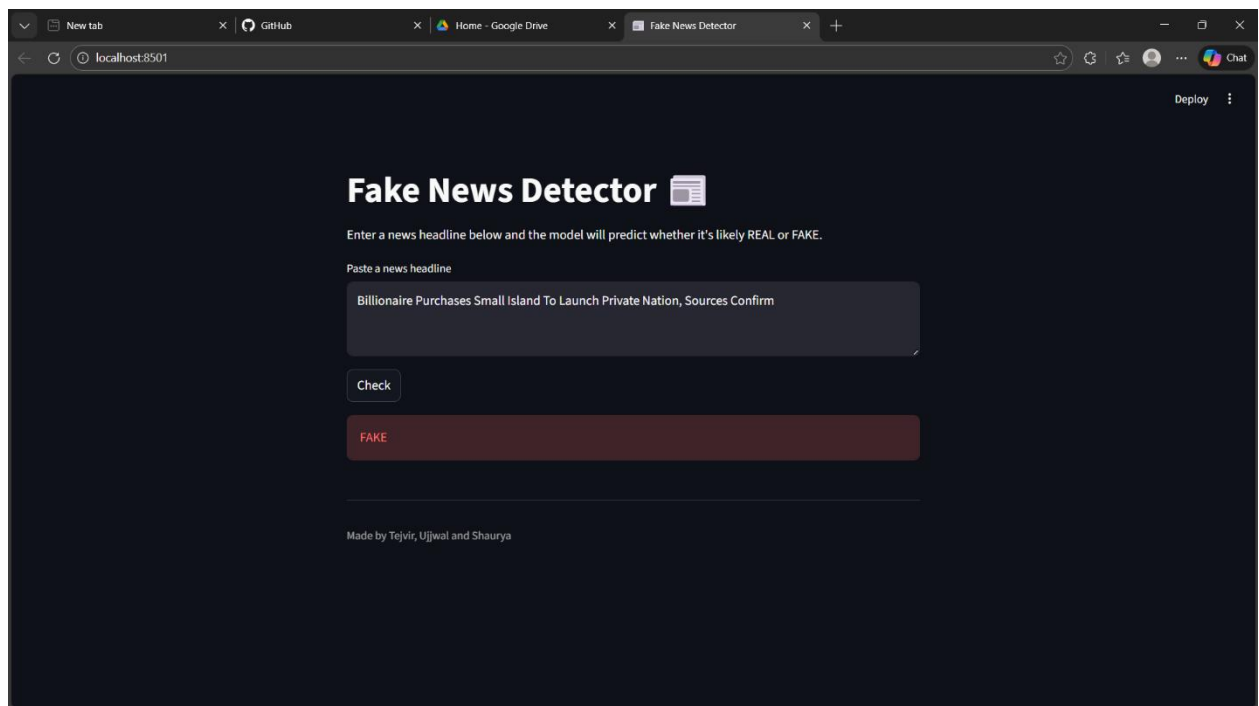
Screenshot showing example predictions such as “FAKE” or “REAL” for given input text.

9. Working Prototype Demonstration

A working prototype of the fake news detection system was implemented where a user can input a news headline or article, and the trained BERT model predicts whether the news is Fake or Real in real time.



Screenshot of Prediction: Real News



Screenshot of Prediction: Fake News

Screenshots showing the user input and corresponding model prediction

10. Code

Setup

Installing Required Libraries

```
!pip install -q transformers datasets scikit-learn torch
```

Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipython-input-1408506528.py in <cell line: 0>()
      1 from google.colab import drive
----> 2 drive.mount('/content/drive')
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/drive.py in
mount(mountpoint, force_remount, timeout_ms, readonly)
    95 def mount(mountpoint, force_remount=False, timeout_ms=120000,
readonly=False):
    96     """Mount your Google Drive at the specified mountpoint path."""
--> 97     return _mount(
    98         mountpoint,
    99         force_remount=force_remount,
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/drive.py in
_mount(mountpoint, force_remount, timeout_ms, ephemeral, readonly)
    132 )
    133 if ephemeral:
--> 134     _message.blocking_request(
```

```

135         'request_auth',
136         request={'authType': 'dfs_ephemeral'},

/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in
blocking_request(request_type, request, timeout_sec, parent)
    174         request_type, request, parent=parent, expect_reply=True
    175     )
--> 176     return read_reply_from_input(request_id, timeout_sec)

/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in
read_reply_from_input(message_id, timeout_sec)
    94     reply = _read_next_input_message()
    95     if reply == _NOT_READY or not isinstance(reply, dict):
---> 96         time.sleep(0.025)
    97         continue
    98     if (

```

KeyboardInterrupt:

Import Libraries

```

import os
import re
import random
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

import torch
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
get_linear_schedule_with_warmup, set_seed
from torch.optim import AdamW

```

Configuration and Seed Setup

```

MODEL_NAME = "bert-base-uncased"
MAX_LEN = 64          # can increase to 128 but training will be slower /
more VRAM
BATCH_SIZE = 16
EPOCHS = 3
LR = 2e-5
SEED = 42

set_seed(SEED)
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

```

Data Loading and Preprocessing

Load Datasets and Initial Cleaning

Change these paths if needed

fake_path = "/content/drive/MyDrive/project_data/Fake.csv"

real_path = "/content/drive/MyDrive/project_data/True.csv"

```
if not os.path.exists(fake_path) or not os.path.exists(real_path):
    raise FileNotFoundError("Check Fake.csv and True.csv paths.")
```

fake_df = pd.read_csv(fake_path)

real_df = pd.read_csv(real_path)

fake_df = fake_df.copy()

real_df = real_df.copy()

fake_df["label"] = 0 *# fake*

real_df["label"] = 1 *# real*

data = pd.concat([fake_df, real_df], ignore_index=True)

print("Combined shape:", data.shape)

Choose which text column to use

text_cols = ["title", "text", "headline", "article"]

text_col = next((c for c in text_cols if c in data.columns), None)

if text_col is None:

```
    raise ValueError(f"No suitable text column found. Columns:
{list(data.columns)}")
```

print("Using text column:", text_col)

def clean_text(s):

if pd.isna(s):

return ""

s = str(s)

light cleaning - DO NOT stem/strip for BERT

s = re.sub(r"\s+", " ", s).strip()

return s

data["text"] = data[text_col].apply(clean_text)

data = data[["text", "label"]]

Train / val / test split: 64 / 16 / 20

train_val_df, test_df = train_test_split(

data, test_size=0.20, random_state=SEED, stratify=data["label"]

)

train_df, val_df = train_test_split(

train_val_df, test_size=0.20, random_state=SEED,

stratify=train_val_df["label"]

)

print("Train:", len(train_df), " Val:", len(val_df), " Test:", len(test_df))

train_df.head()

Combined shape: (44898, 5)
Using text column: title
Train: 28734 Val: 7184 Test: 8980

```
{"summary": "{\n  \"name\": \"train_df\", \n  \"rows\": 28734, \n  \"fields\": [\n    {\n      \"column\": \"text\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 26160, \n        \"samples\": [\n          \"AWESOME MESSAGE TO NEVER-TRUMPERS: \\\u201cKiss and make up!\\\u201d\\\u2026\\\u201dThe people have chosen!\\\u201d\", \n          \"Trump escalates attacks on judge in Trump University suit: WSJ\", \n          \"WILLIAM SHATNER Blasts \\\u2018Social Justice Warriors\\\u2019 Who Criticize Trump\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"label\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 0, \n        \"max\": 1, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          0, \n          1 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    ] \n  }\", "type": "dataframe", "variable_name": "train_df"}
```

Tokenization and Dataset Preparation

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

```
class NewsDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len):
        self.texts = list(texts)
        self.labels = list(labels)
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = int(self.labels[idx])

        enc = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_len,
            return_tensors="pt"
        )

        item = {
            "input_ids": enc["input_ids"].squeeze(0),
            "attention_mask": enc["attention_mask"].squeeze(0),
            "labels": torch.tensor(label, dtype=torch.long)
        }
        return item
```

train_dataset = NewsDataset(train_df["text"], train_df["label"], tokenizer, MAX_LEN)

val_dataset = NewsDataset(val_df["text"], val_df["label"], tokenizer,

```

MAX_LEN)
test_dataset = NewsDataset(test_df["text"], test_df["label"], tokenizer,
MAX_LEN)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE,
shuffle=False)

len(train_loader), len(val_loader), len(test_loader)

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94:
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings
tab (https://huggingface.co/settings/tokens), set it as secret in your Google
Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(

{"model_id":"1459c2c4ff5b493f925fe7c9a209b59c","version_major":2,"version_min
or":0}

{"model_id":"1c4a0edf8f794eb88516a29346c7ae95","version_major":2,"version_min
or":0}

{"model_id":"39cdceb41da245b388afc52bb8ac3e78","version_major":2,"version_min
or":0}

{"model_id":"97071b60f19b44b1be080f0becf6cc2a","version_major":2,"version_min
or":0}

(1796, 449, 562)

```

Model Initialization and Training

```

model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME,
num_labels=2)
model.to(device)

print("Model loaded on", device)

{"model_id":"8b38d6ebcfa340f197c4116932f20ff5","version_major":2,"version_min
or":0}

```

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized:
['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Model loaded on cuda

Define Optimizer, Scheduler, Loss and Training Loop

```
optimizer = AdamW(model.parameters(), lr=LR)

total_steps = len(train_loader) * EPOCHS
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=int(0.1 * total_steps),
    num_training_steps=total_steps
)

loss_fn = torch.nn.CrossEntropyLoss()

def train_one_epoch(epoch):
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    for batch in train_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        optimizer.zero_grad()

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        loss = outputs.loss
        logits = outputs.logits

        loss.backward()
        optimizer.step()
        scheduler.step()

        total_loss += loss.item() * input_ids.size(0)
        preds = torch.argmax(logits, dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    avg_loss = total_loss / total
    acc = correct / total
    print(f"Epoch {epoch}: Train loss={avg_loss:.4f}, acc={acc:.4f}")
    return avg_loss, acc

def eval_model():
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
```

```

    for batch in val_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        loss = outputs.loss
        logits = outputs.logits

        total_loss += loss.item() * input_ids.size(0)
        preds = torch.argmax(logits, dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    avg_loss = total_loss / total
    acc = correct / total
    print(f"Val    : loss={avg_loss:.4f}, acc={acc:.4f}")
    return avg_loss, acc

for epoch in range(1, EPOCHS + 1):
    train_one_epoch(epoch)
    eval_model()

```

```

Epoch 1: Train loss=0.1277, acc=0.9508
Val    : loss=0.0496, acc=0.9825
Epoch 2: Train loss=0.0253, acc=0.9913
Val    : loss=0.0397, acc=0.9861
Epoch 3: Train loss=0.0069, acc=0.9978
Val    : loss=0.0455, acc=0.9887

```

Test Set Evaluation

```

model.eval()
all_labels = []
all_preds = []

with torch.no_grad():
    for batch in test_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )
        logits = outputs.logits
        preds = torch.argmax(logits, dim=1)

        all_labels.extend(labels.cpu().numpy().tolist())
        all_preds.extend(preds.cpu().numpy().tolist())

```



```
test_acc = accuracy_score(all_labels, all_preds)
print("Test accuracy:", test_acc)
print("\nClassification report:\n")
print(classification_report(all_labels, all_preds, digits=4))
```

Test accuracy: 0.9907572383073496

Classification report:

	precision	recall	f1-score	support
0	0.9919	0.9904	0.9912	4696
1	0.9895	0.9911	0.9903	4284
accuracy			0.9908	8980
macro avg	0.9907	0.9908	0.9907	8980
weighted avg	0.9908	0.9908	0.9908	8980

Model Persistence

Create Save Directory

```
SAVE_DIR = "/content/drive/MyDrive/project_data/modelnew22:39and20"
```

```
import os
```

```
os.makedirs(SAVE_DIR, exist_ok=True)
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-775340475.py in <cell line: 0>()
      1 SAVE_DIR = "/content/drive/MyDrive/project_data/modelnew22:39and20"
----> 2 os.makedirs(SAVE_DIR, exist_ok=True)
```

NameError: name 'os' is not defined

Save Model and Tokenizer

```
# Save model weights, config, tokenizer files
```

```
model.save_pretrained(SAVE_DIR)
```

```
tokenizer.save_pretrained(SAVE_DIR)
```

```
print("Model & tokenizer saved to:", SAVE_DIR)
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-2610799206.py in <cell line: 0>()
      1 # Save model weights, config, tokenizer files
----> 2 model.save_pretrained(SAVE_DIR)
      3 tokenizer.save_pretrained(SAVE_DIR)
      4
      5 print("Model & tokenizer saved to:", SAVE_DIR)
```

NameError: name 'model' is not defined

Model Inference

Load Saved Model and Tokenizer

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
```

```
LOAD_DIR = "/content/drive/MyDrive/project_data/modelnew22:39and20"
```

```
tokenizer = AutoTokenizer.from_pretrained(LOAD_DIR)
```

```
model = AutoModelForSequenceClassification.from_pretrained(LOAD_DIR)
```

```
model.to("cuda")
```

```
model.eval()
```

```
BertForSequenceClassification(
```

```
  (bert): BertModel(
```

```
    (embeddings): BertEmbeddings(
```

```
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
```

```
      (position_embeddings): Embedding(512, 768)
```

```
      (token_type_embeddings): Embedding(2, 768)
```

```
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
```

```
      (dropout): Dropout(p=0.1, inplace=False)
```

```
    )
```

```
    (encoder): BertEncoder(
```

```
      (layer): ModuleList(
```

```
        (0-11): 12 x BertLayer(
```

```
          (attention): BertAttention(
```

```
            (self): BertSdpaSelfAttention(
```

```
              (query): Linear(in_features=768, out_features=768, bias=True)
```

```
              (key): Linear(in_features=768, out_features=768, bias=True)
```

```
              (value): Linear(in_features=768, out_features=768, bias=True)
```

```
              (dropout): Dropout(p=0.1, inplace=False)
```

```
            )
```

```
            (output): BertSelfOutput(
```

```
              (dense): Linear(in_features=768, out_features=768, bias=True)
```

```
              (LayerNorm): LayerNorm((768,), eps=1e-12,
```

```
elementwise_affine=True)
```

```
              (dropout): Dropout(p=0.1, inplace=False)
```

```
            )
```

```
          )
```

```
          (intermediate): BertIntermediate(
```

```
            (dense): Linear(in_features=768, out_features=3072, bias=True)
```

```
            (intermediate_act_fn): GELUActivation()
```

```
          )
```

```
          (output): BertOutput(
```

```
            (dense): Linear(in_features=3072, out_features=768, bias=True)
```

```
            (LayerNorm): LayerNorm((768,), eps=1e-12,
```

```
elementwise_affine=True)
```

```
            (dropout): Dropout(p=0.1, inplace=False)
```

```
          )
```

```
        )
```

```
      )
```

```
    )
```

```
    (pooler): BertPooler(
```

```
      (dense): Linear(in_features=768, out_features=768, bias=True)
```

```
      (activation): Tanh()
```

```
    )
```

```
  )
```

```

(dropout): Dropout(p=0.1, inplace=False)
(classifier): Linear(in_features=768, out_features=2, bias=True)
)

```

Single Prediction Function

```

def predict_news(text):
    enc = tokenizer(
        text,
        truncation=True,
        padding="max_length",
        max_length=64,
        return_tensors="pt"
    )

    input_ids = enc["input_ids"].to("cuda")
    attention_mask = enc["attention_mask"].to("cuda")

    with torch.no_grad():
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        pred = torch.argmax(logits, dim=1).item()

    return "FAKE" if pred == 0 else "REAL"

```

Example Predictions

```

print(predict_news(" DONALD TRUMP DEAD"))
print(predict_news("Princess Diana enters Paris waxwork museum in revenge
dress"))

```

```

FAKE
REAL

```

Data Exploration and Visualization

Confusion Matrix Plot

```

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

def plot_confusion(test_loader, model, device):
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for batch in test_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            outputs = model(input_ids=input_ids,
attention_mask=attention_mask)
            preds = torch.argmax(outputs.logits, dim=1)

```

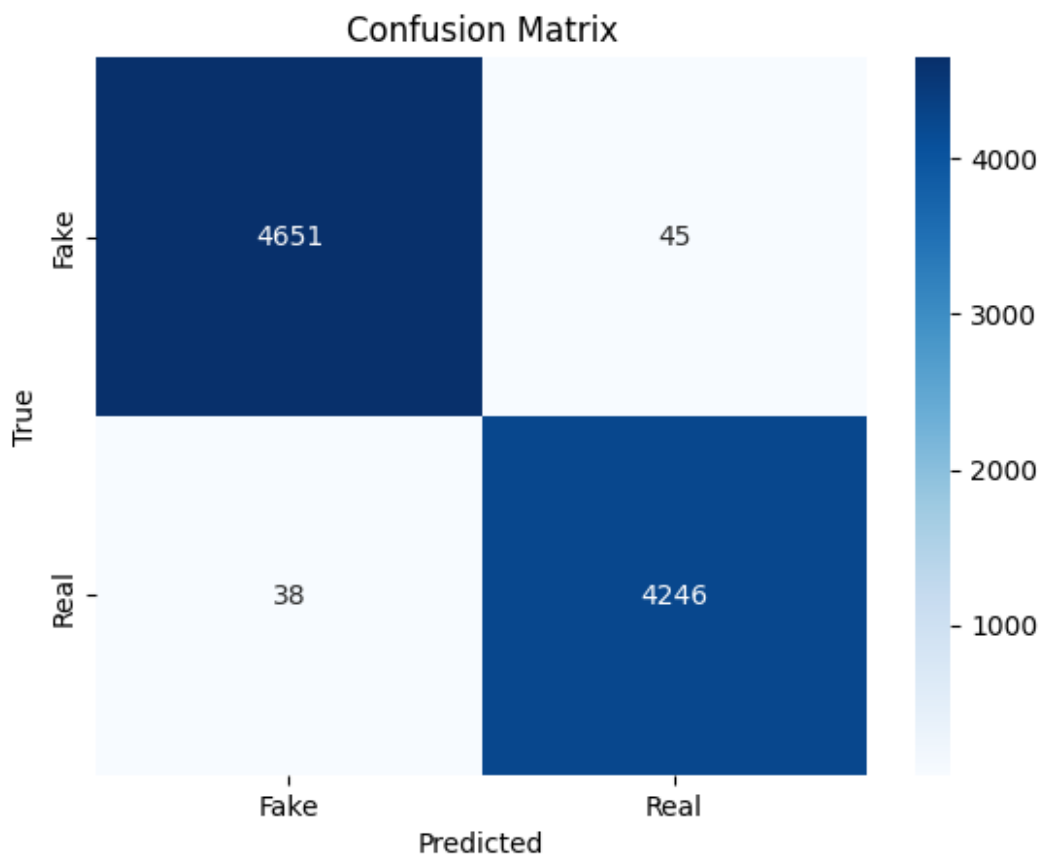
```

all_preds.extend(preds.cpu().numpy())
all_labels.extend(labels.cpu().numpy())

cm = confusion_matrix(all_labels, all_preds)
sns.heatmap(cm, annot=True, cmap="Blues", fmt="d",
            xticklabels=["Fake", "Real"],
            yticklabels=["Fake", "Real"])
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()

plot_confusion(test_loader, model, device)

```



Data Overview (After Training Context)

```

# After you have already created `data` from Fake.csv + True.csv:
# fake_df['label']=0; real_df['label']=1; data = concat(...)

```

```

df = data.copy() # American fake/true dataset
df.head()

```

```

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 44898,\n  \"fields\": [\n    {\n      \"column\": \"text\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 38726,\n        \"samples\": [\n          \"DELAY OF RELEASE OF CLINTON SCHEDULES Until After Election Could Backfire\\u2026Feds Out Of Control\\u2026\",\n          \"Turkish mayor says forced to quit in Erdogan purge of local government\\u2026\",\n          \"Ranting Trump Accidentally Confirms He Is Under Investigation After Staff Claimed It Was

```

```

Fake News\\",\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          },\n          {\n          \"column\": \"label\", \n          \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 0, \n          \"min\": 0, \n          \"max\": 1, \n          \"num_unique_values\": 2, \n          \"samples\": [\n          1, \n          0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n          } \n          ] \n          }\", \"type\": \"dataframe\", \"variable_name\": \"df\"}

```

Label Distribution Plot

```
import matplotlib.pyplot as plt
```

```

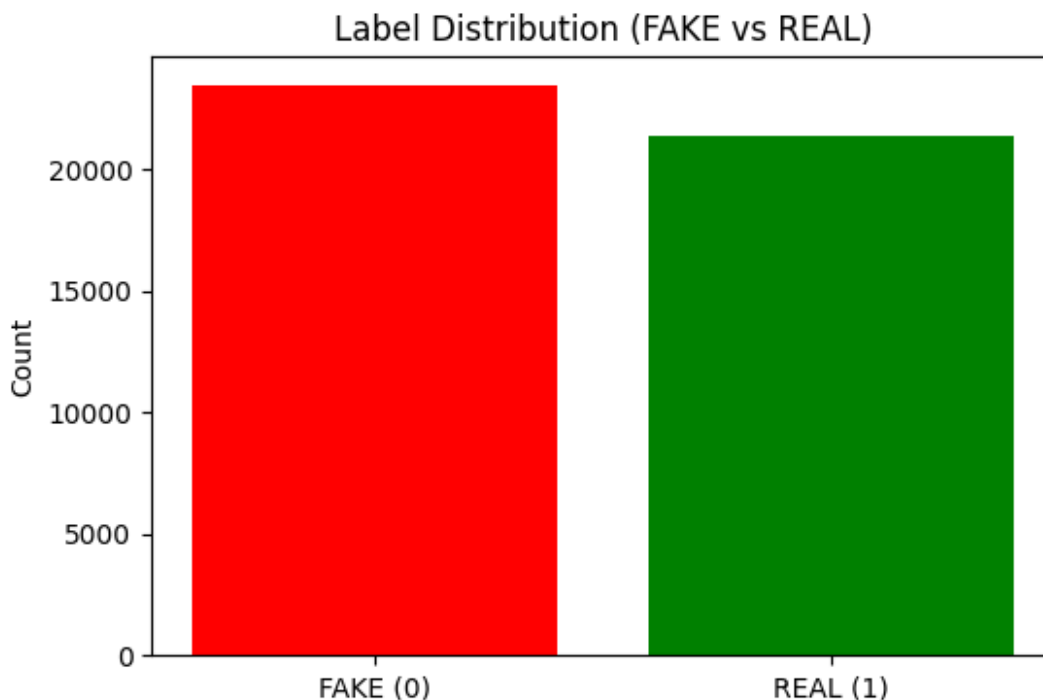
label_counts = df[\"label\"].value_counts().sort_index()
labels = [\"FAKE (0)\", \"REAL (1)\"]

```

```

plt.figure(figsize=(6, 4))
plt.bar(labels, label_counts, color=[\"red\", \"green\"])
plt.title(\"Label Distribution (FAKE vs REAL)\")
plt.ylabel(\"Count\")
plt.show()

```



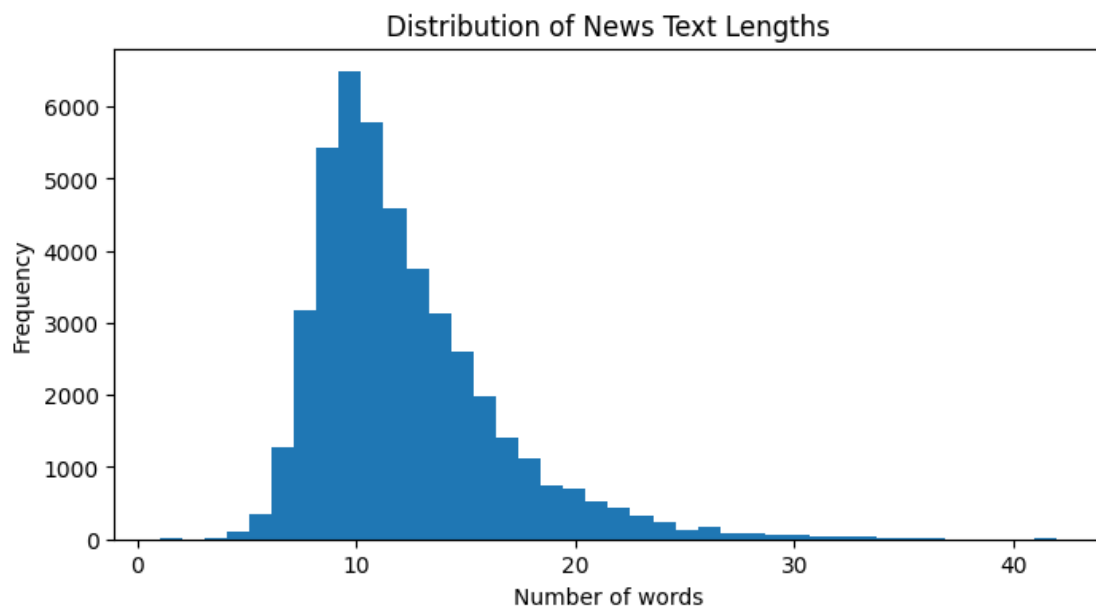
Text Length Distribution

```
df[\"length\"] = df[\"text\"].apply(lambda x: len(str(x).split()))
```

```

plt.figure(figsize=(8, 4))
plt.hist(df[\"length\"], bins=40)
plt.title(\"Distribution of News Text Lengths\")
plt.xlabel(\"Number of words\")
plt.ylabel(\"Frequency\")
plt.show()

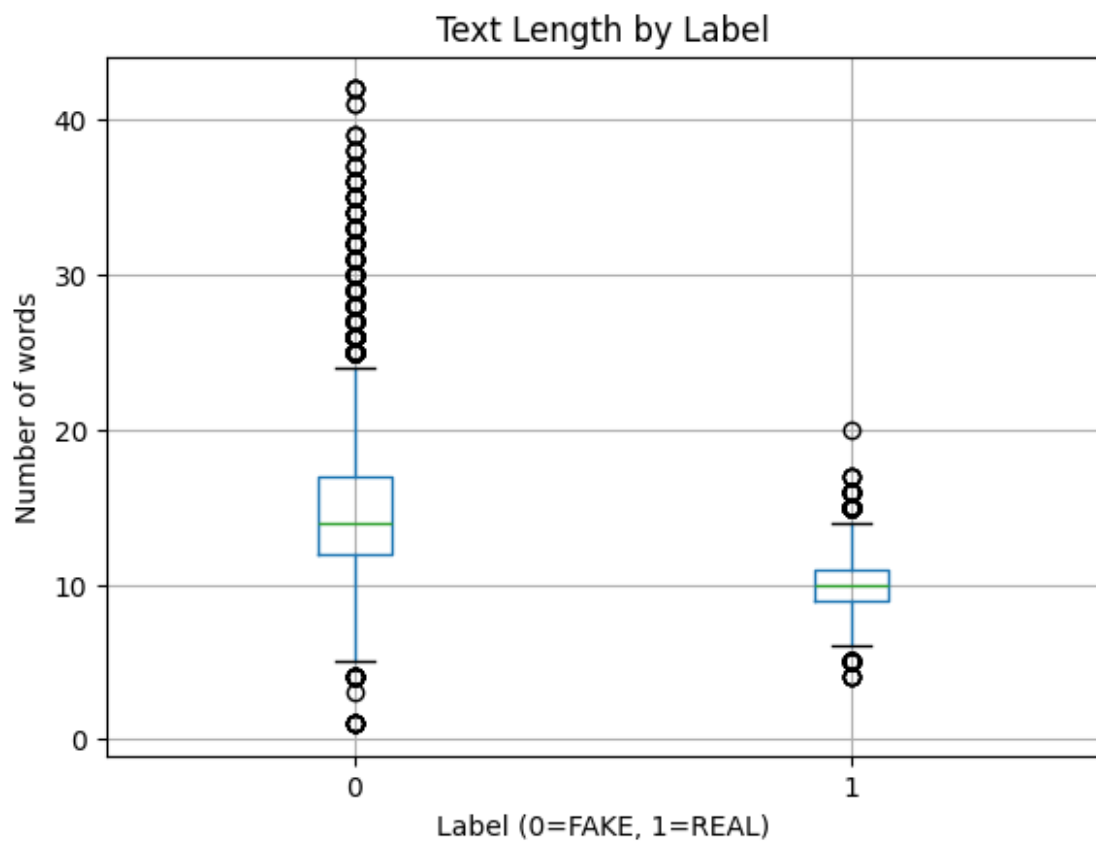
```



Text Length by Label Box Plot

```
plt.figure(figsize=(6,4))
df.boxplot(column="length", by="label")
plt.title("Text Length by Label")
plt.suptitle("")
plt.xlabel("Label (0=FAKE, 1=REAL)")
plt.ylabel("Number of words")
plt.show()
```

<Figure size 600x400 with 0 Axes>



```
!pip install -q wordcloud
from wordcloud import WordCloud
```

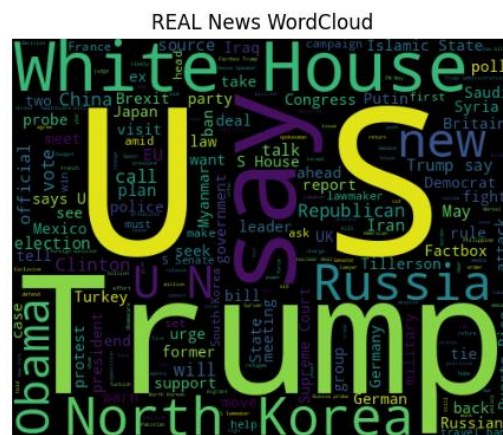
```
fake_text = " ".join(df[df["label"]==0]["text"].astype(str).tolist())
real_text = " ".join(df[df["label"]==1]["text"].astype(str).tolist())
```

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
plt.imshow(WordCloud(width=500, height=400).generate(fake_text))
plt.title("FAKE News WordCloud")
plt.axis("off")
```

```
plt.subplot(1,2,2)
plt.imshow(WordCloud(width=500, height=400).generate(real_text))
plt.title("REAL News WordCloud")
plt.axis("off")
```

```
plt.show()
```



Further Evaluation and Analysis

```
def train_one_epoch(epoch):
```

```
...
return avg_loss, acc
```

```
def eval_model():
```

```
...
return avg_loss, acc
```

Classification Report (Test Set)

```
from sklearn.metrics import classification_report
```

```
print(classification_report(all_labels, all_preds, target_names=["FAKE",  
"REAL"]))
```

	precision	recall	f1-score	support
FAKE	0.99	0.99	0.99	4696
REAL	0.99	0.99	0.99	4284

accuracy			0.99	8980
macro avg	0.99	0.99	0.99	8980
weighted avg	0.99	0.99	0.99	8980

Correct vs. Wrong Predictions Examples

```
test_df = test_df.reset_index(drop=True)
test_df["true_label"] = all_labels
test_df["pred_label"] = all_preds
```

```
correct = test_df[test_df["true_label"] == test_df["pred_label"]].head(10)
wrong = test_df[test_df["true_label"] != test_df["pred_label"]].head(10)
```

```
print("\n☑ Correct predictions (first 10):\n")
for _, row in correct.iterrows():
    print("TEXT :", row["text"][:120].replace("\n", " ") + "...")
    print("TRUE :", "REAL" if row["true_label"]==1 else "FAKE",
          "| PRED:", "REAL" if row["pred_label"]==1 else "FAKE")
    print("-"*80)

print("\n✗ Wrong predictions (first 10):\n")
for _, row in wrong.iterrows():
    print("TEXT :", row["text"][:120].replace("\n", " ") + "...")
    print("TRUE :", "REAL" if row["true_label"]==1 else "FAKE",
          "| PRED:", "REAL" if row["pred_label"]==1 else "FAKE")
    print("-"*80)
```

☑ Correct predictions (first 10):

```
TEXT : Trump names Don McGahn as White House Counsel - transition official...
TRUE : REAL | PRED: REAL
-----
---
TEXT : Senate Judiciary Committee seeks Comey memos, possible Trump tapes...
TRUE : REAL | PRED: REAL
-----
---
TEXT : South Carolina Governor Nikki Haley Says We've Never Passed Racist
Laws In This Country...
TRUE : FAKE | PRED: FAKE
-----
---
TEXT : China's internet regulator denounces its former chief now under
investigation...
TRUE : REAL | PRED: REAL
-----
---
TEXT : Minsk cultural hub becomes haven from authorities...
TRUE : REAL | PRED: REAL
-----
---
TEXT : Trump Is Going To Make Measles Great Again, Will Meet With Top Anti-
```


Vaxxer To Discuss Vaccines...

TRUE : FAKE | PRED: FAKE

TEXT : Trump unveils companies' \$500 million U.S. drug packaging project...

TRUE : REAL | PRED: REAL

TEXT : YEAR IN REVIEW: 2017 Top Ten Conspiracies...

TRUE : FAKE | PRED: FAKE

TEXT : WHO urges Trump to expand Obamacare, ensure healthcare for all...

TRUE : REAL | PRED: REAL

TEXT : Clinton would stick with Garland as nominee for Supreme Court: Reid...

TRUE : REAL | PRED: REAL

✗ Wrong predictions (first 10):

TEXT : Bipartisan Harvard panel recommends hacking safeguards for elections...

TRUE : REAL | PRED: FAKE

TEXT : Texas politician slammed for tweet sent after Florida shooting...

TRUE : REAL | PRED: FAKE

TEXT : Republicans Block Young Voters From Participating In Ohio Primary...

TRUE : FAKE | PRED: REAL

TEXT : Trump flag-burning tweet leads activists to burn some flags in New York...

TRUE : REAL | PRED: FAKE

TEXT : Trump draws full house at own event as he snubs Fox News debate...

TRUE : REAL | PRED: FAKE

TEXT : Trump-Supporting Republican Candidate Wishes Rape Upon Female Reporter...

TRUE : FAKE | PRED: REAL

TEXT : Good News for Silver in 2017...

TRUE : FAKE | PRED: REAL

```
TEXT : Trump Dominates South Carolina, Dooming The Republican Party...
TRUE : FAKE | PRED: REAL
```

```
-----
---
```

```
TEXT : Republican Ryan is raising big money but not for White House race...
TRUE : REAL | PRED: FAKE
```

```
-----
---
```

```
TEXT : For Islamophobia, take two of these and call a Muslim in the
morning...
TRUE : REAL | PRED: FAKE
```

```
-----
---
```

Collect Predictions, Probabilities, and Labels (Initial)

```
# Collect predictions, probabilities and labels
```

```
model.eval()
```

```
all_labels = []
```

```
all_preds = []
```

```
prob_list = [] # <---- THIS is what you were missing
```

```
with torch.no_grad():
```

```
    for batch in test_loader:
```

```
        input_ids = batch["input_ids"].to(device)
```

```
        attention_mask = batch["attention_mask"].to(device)
```

```
        labels = batch["labels"].to(device)
```

```
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
```

```
        logits = outputs.logits
```

```
        # prediction
```

```
        preds = torch.argmax(logits, dim=1)
```

```
        all_preds.extend(preds.cpu().numpy())
```

```
        # store labels
```

```
        all_labels.extend(labels.cpu().numpy())
```

```
        # store probabilities for REAL class (1)
```

```
        probs = torch.softmax(logits, dim=1)
```

```
        real_probs = probs[:, 1].cpu().numpy()
```

```
        prob_list.extend(real_probs.tolist())
```

```
# probability of REAL news
```

```
# <---- saved here
```

Refined Collection of Predictions and Probabilities

```
# Collect predictions, probabilities and labels from TEST SET
```

```
model.eval()
```

```
all_labels = []
```

```
all_preds = []
```

```
prob_list = [] # prob of REAL (class 1)
```

```
with torch.no_grad():
```

```
    for batch in test_loader:
```

```
        input_ids = batch["input_ids"].to(device)
```

```

attention_mask = batch["attention_mask"].to(device)
labels = batch["labels"].to(device)

outputs = model(input_ids=input_ids, attention_mask=attention_mask)
logits = outputs.logits

# predicted class
preds = torch.argmax(logits, dim=1)
all_preds.extend(preds.cpu().numpy())

# true labels
all_labels.extend(labels.cpu().numpy())

# probability of REAL (class 1)
probs = torch.softmax(logits, dim=1)[: , 1] # shape: [batch]
prob_list.extend(probs.cpu().numpy().tolist())

len(all_labels), len(all_preds), len(prob_list)

(8980, 8980, 8980)

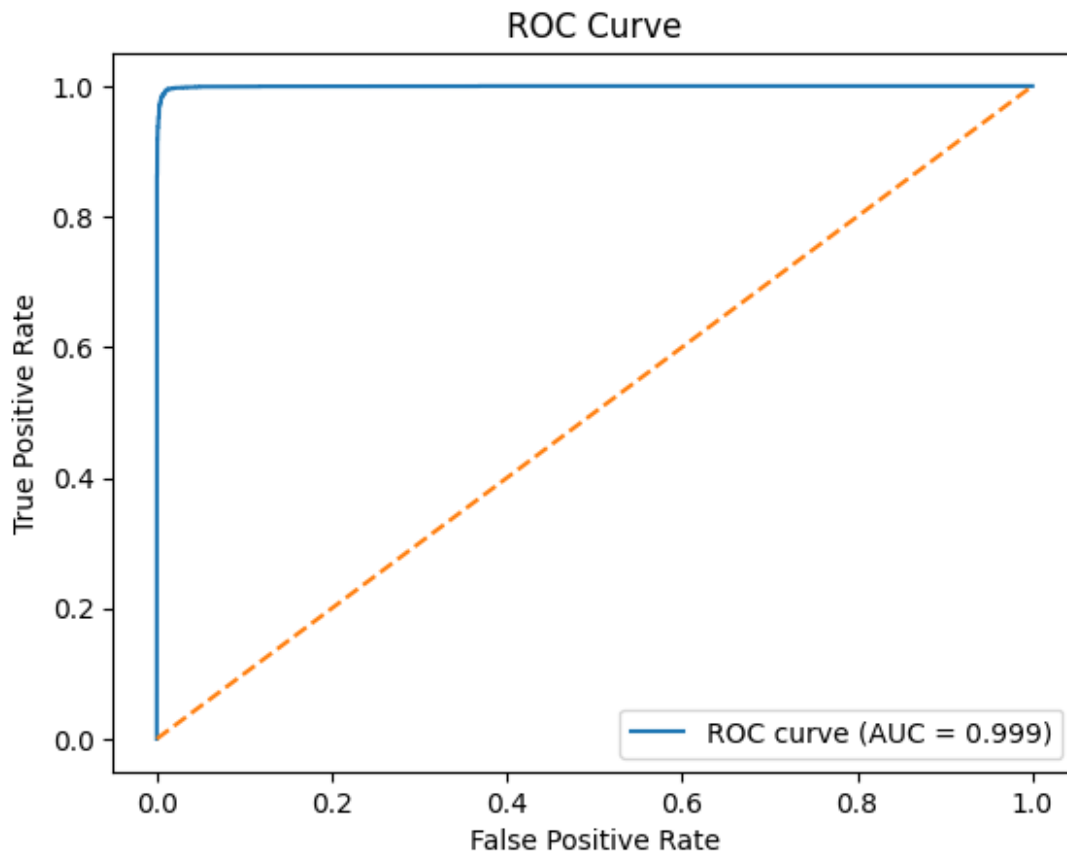
ROC Curve and AUC Score
from sklearn.metrics import roc_curve, roc_auc_score
import numpy as np
import matplotlib.pyplot as plt

probs = np.array(prob_list) # 1-D vector of P(REAL)

fpr, tpr, _ = roc_curve(all_labels, probs)
auc_score = roc_auc_score(all_labels, probs)

plt.plot(fpr, tpr, label=f"ROC curve (AUC = {auc_score:.3f})")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

```



Comprehensive Model Performance Metrics and Plots

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    precision_recall_curve,
    average_precision_score,
)

# ===== 1. Collect labels, predictions, probabilities =====
model.eval()

all_labels = []
all_preds = []
prob_list = []  # probability of class 1 (REAL)

with torch.no_grad():
    for batch in test_loader:
        input_ids = batch["input_ids"].to(device)
```

```

attention_mask = batch["attention_mask"].to(device)
labels = batch["labels"].to(device)

outputs = model(input_ids=input_ids, attention_mask=attention_mask)
logits = outputs.logits

probs = torch.softmax(logits, dim=1)          # shape [batch, 2]
real_probs = probs[:, 1]                     # P(class = REAL)

preds = torch.argmax(logits, dim=1)

all_labels.extend(labels.cpu().numpy())
all_preds.extend(preds.cpu().numpy())
prob_list.extend(real_probs.cpu().numpy())

all_labels = np.array(all_labels)
all_preds = np.array(all_preds)
prob_list = np.array(prob_list)  # shape (N,) <-- this is correct

# ===== 2. Basic metrics =====
acc = accuracy_score(all_labels, all_preds)
prec = precision_score(all_labels, all_preds)
rec = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)

print("Accuracy :", acc)
print("Precision:", prec)
print("Recall   :", rec)
print("F1 score :", f1)
print("\nClassification report:\n")
print(classification_report(all_labels, all_preds, target_names=["FAKE",
"REAL"]))

# ===== 3. Confusion matrix =====
cm = confusion_matrix(all_labels, all_preds)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d",
            xticklabels=["Pred FAKE", "Pred REAL"],
            yticklabels=["True FAKE", "True REAL"],
            cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ===== 4. ROC curve + AUC (THIS IS WHERE YOUR BUG WAS) =====
# y_score must be 1-D: probability of the positive class (REAL = 1)
fpr, tpr, _ = roc_curve(all_labels, prob_list)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(5,4))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0, 1], [0, 1], linestyle="--")

```

```

plt.title("ROC Curve")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

# ===== 5. Precision-Recall curve =====
precisions, recalls, _ = precision_recall_curve(all_labels, prob_list)
ap = average_precision_score(all_labels, prob_list)

plt.figure(figsize=(5,4))
plt.plot(recalls, precisions, label=f"AP = {ap:.3f}")
plt.title("Precision-Recall Curve")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.legend()
plt.show()

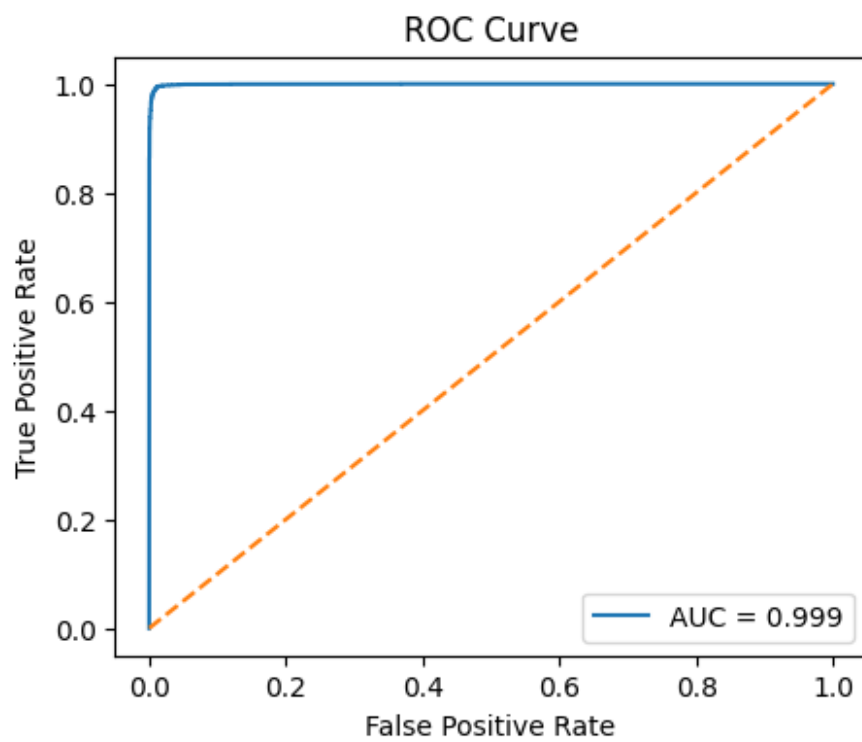
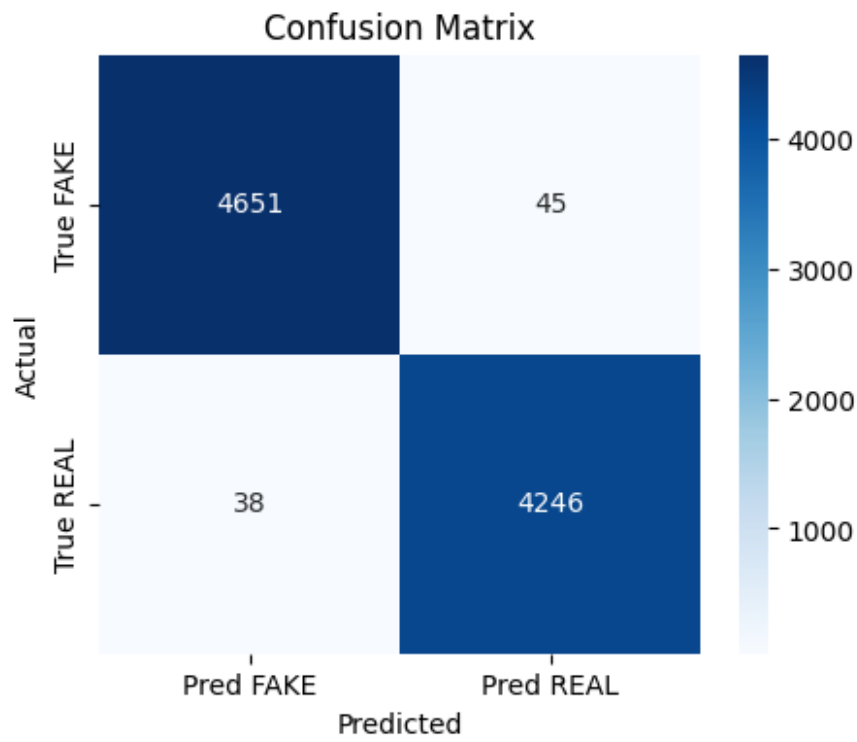
# ===== 6. Probability histogram =====
plt.figure(figsize=(6,4))
plt.hist(prob_list[all_labels == 0], bins=20, alpha=0.5, label="True FAKE")
plt.hist(prob_list[all_labels == 1], bins=20, alpha=0.5, label="True REAL")
plt.xlabel("Predicted probability of REAL (class 1)")
plt.ylabel("Count")
plt.title("Distribution of Predicted Probabilities")
plt.legend()
plt.show()

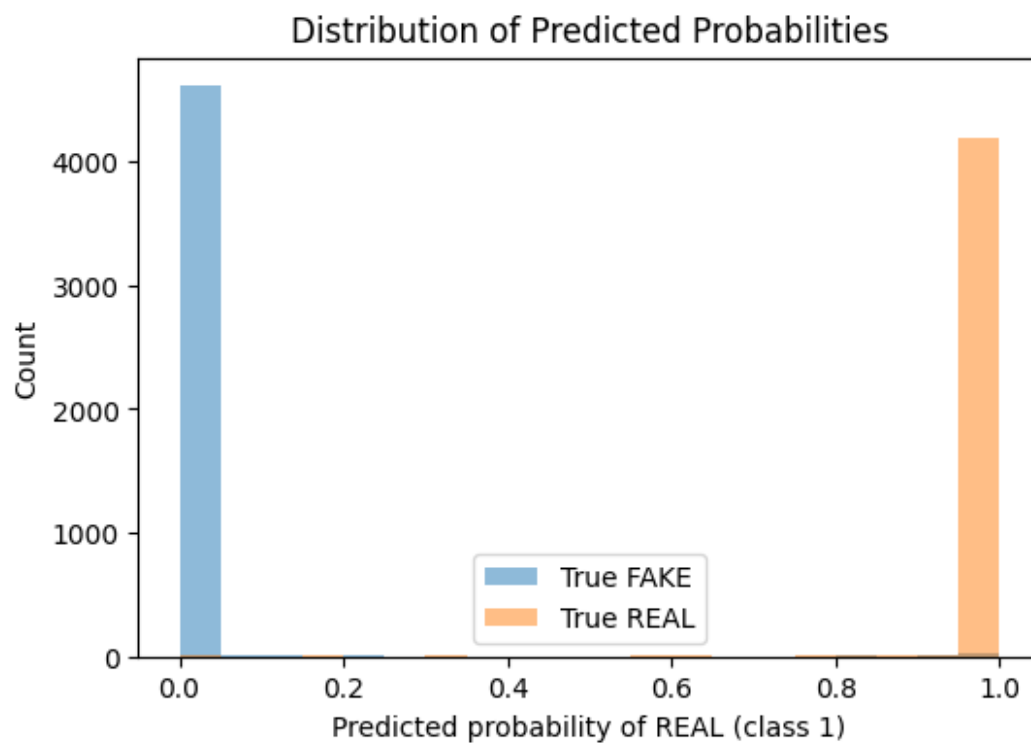
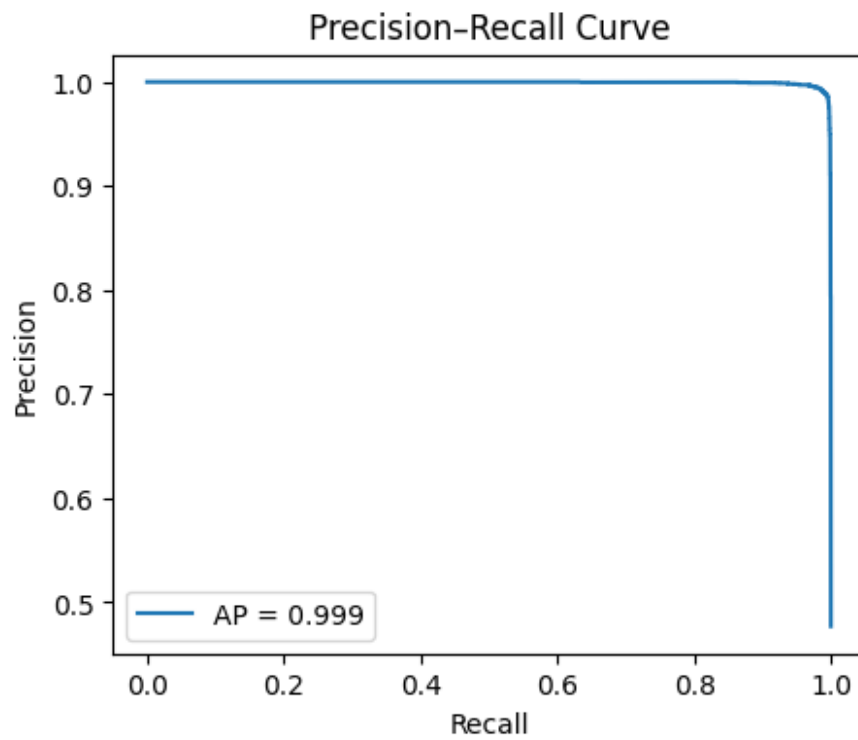
Accuracy : 0.9907572383073496
Precision: 0.9895129340480074
Recall    : 0.9911297852474323
F1 score  : 0.9903206997084548

```

Classification report:

	precision	recall	f1-score	support
FAKE	0.99	0.99	0.99	4696
REAL	0.99	0.99	0.99	4284
accuracy			0.99	8980
macro avg	0.99	0.99	0.99	8980
weighted avg	0.99	0.99	0.99	8980





End of Notebook

11. Conclusion

This project successfully demonstrates the application of a transformer-based deep learning model for fake news detection. The BERT model effectively captures contextual information from news articles, resulting in high classification accuracy and robust performance across multiple evaluation metrics.

The results confirm that modern NLP techniques can play a crucial role in combating misinformation. Future enhancements may include implementing multilingual models, adding source credibility features, and deploying the model as a web application.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, Minneapolis, MN, USA, pp. 4171–4186, 2019.
- [2] C. Bisailon, “Fake and Real News Dataset,” Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>. [Accessed: Aug. 2025].
- [3] Hugging Face, “Transformers: State-of-the-art Natural Language Processing,” 2024. [Online]. Available: <https://huggingface.co/docs/transformers>. [Accessed: Aug. 2025].
- [4] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.
- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” *Proceedings of the International Conference on Learning Representations (ICLR)*, 2013.